

Diseño del Tema 04 — Aspectos Avanzados de Programación Funcional

ESTADO: APROBADO — Rediseñado a partir de filminas aprobadas (F-01 a F-36)

Duración de clase: 120 minutos Enfoque: profundizar en programación funcional con implementaciones reales en Clojure y TypeScript. Cuatro ejes: (1) HOF y composición, (2) partial/currying y validación web con `Result`, (3) recursión de cola y patrones avanzados (memoization, lazy, DSLs), (4) taller integrador comparativo. Se excluye concurrencia.

1. Objetivos de aprendizaje (alineados a filminas aprobadas)

1. **OA-1 — First-class functions y HOF (F-04, F-05):** Comprender que las funciones son valores y que `map`, `filter`, `reduce`, `compose`, `pipe` son HOF fundamentales.
2. **OA-2 — Composición y pipelines (F-06 a F-08):** Construir pipelines de transformación de datos usando `pipe` en TS y `->>` en Clojure sin variables intermedias.
3. **OA-3 — Inmutabilidad y transformación declarativa (F-09 a F-12):** Aplicar spread/inmutabilidad y usar `map`, `filter`, `flatMap`, `reduce` para transformar colecciones.
4. **OA-4 — Partial application y currying (F-13 a F-18):** Diferenciar partial application de currying, implementar fábricas de funciones en ambos lenguajes.
5. **OA-5 — Modelado de errores con Result (F-19 a F-21):** Reemplazar excepciones por `Result<T, E>`, encadenar validaciones tipadas, usar en handlers HTTP.
6. **OA-6 — Middleware como HOF (F-22, F-23):** Construir middlewares composables (auth, logging) aplicando partial application y `pipe`.
7. **OA-7 — Recursión de cola (F-24 a F-27):** Explicar el problema del stack overflow, aplicar TCO con `recur` en Clojure y acumuladores en TS.
8. **OA-8 — Patrones avanzados (F-28 a F-31):** Implementar `memoize`, entender lazy sequences y construir mini-DSLs data-driven en Clojure.
9. **OA-9 — Transferencia de patrones (F-32):** Mapear cada patrón entre Clojure y TypeScript reconociendo que la idea es portable.
10. **OA-10 — Aplicación integrada (F-33):** Resolver un problema completo de validación/pipeline sin mutaciones ni excepciones.

2. Prerrequisitos

- Tema 03 aprobado: funciones puras, inmutabilidad, `map` / `filter` / `reduce` básicos.
- Tipos básicos de TypeScript (generics, union types, type aliases).
- Sintaxis elemental de Clojure (`defn`, `let`, `if`, listas y mapas).

3. Alcance del tema (derivado de filminas F-01 a F-36)

Incluye

- First-class functions y HOF: `map`, `filter`, `reduce`, `flatMap` / `mapcat` (F-04 a F-12).
- `compose` y `pipe` en TS; thread macro `->>` en Clojure (F-06 a F-08).
- Inmutabilidad con spread operator y objetos nuevos (F-09).
- Partial application: concepto, `partial` nativo de Clojure, closures en TS (F-13 a F-15).
- Currying: concepto, diferencia con partial, `curry2` genérico en TS, lambdas anidadas en Clojure (F-16 a F-18).
- Tipo `Result<T, E>`: definición, encadenamiento con `chain`, uso en handler HTTP (F-19 a F-21).
- Middleware composable: `withAuth`, `withLogging`, composición con `pipe` (F-22, F-23).
- Recursión de cola: concepto, `recur` en Clojure, acumuladores en TS, trampolining conceptual (F-24 a F-27).
- Memoization: `memoize` en TS y `memoize` nativo de Clojure (F-28, F-29).
- Lazy sequences: `range`, `lazy-seq`, streaming de datos (F-30).
- Mini-DSLs data-driven con HOF en Clojure (F-31).
- Tabla comparativa Clojure ↔ TypeScript (F-32).
- Taller integrador: pipeline de registro de usuarios A (TS) + B (Clojure) (F-33).

No incluye (F-03)

- Concurrency, `core.async`, STM, go-blocks.
- Teoría categórica: funtores, mónadas formales, flechas.
- Librerías FP completas: Ramda, fp-ts, crocks.
- Frameworks async/reactive: RxJS, Effect-TS.

4. Estructura de la clase (120 min — 36 filminas)

Bloque 1 — Funciones de orden superior y composición (35 min, F-01 a F-12)

Filmina	Contenido	Nivel Bloom	Tiempo est.
F-01	Portada	—	1 min
F-02	Alcance: mapa de los 4 bloques	Recordar	2 min
F-03	Fuera de scope	Recordar	2 min
F-04	First-class functions: concepto	Comprender	3 min
F-05	HOF: map, filter, reduce, compose, pipe	Comprender	3 min
F-06	compose vs pipe: definición formal	Comprender	3 min
F-07	pipe en TypeScript: implementación y pipeline <code>normalizeEmail</code>	Aplicar	4 min
F-08	Thread macro <code>->></code> en Clojure: pipeline de pedidos	Aplicar	4 min
F-09	Inmutabilidad en TS: spread operator, <code>normalizeUser</code>	Aplicar	3 min
F-10	map/filter/flatMap en profundidad: firmas y diferencias	Comprender	3 min
F-11	flatMap aplicado: usuarios y permisos, <code>mapcat</code>	Aplicar	3 min
F-12	reduce como fold universal: índice y suma	Aplicar	4 min

Bloque 2 — Aplicación parcial, currying y validación web (35 min, F-13 a F-23)

Filmina	Contenido	Nivel Bloom	Tiempo est.
F-13	Partial application: concepto y uso en web	Comprender	3 min
F-14	Partial en TS: closures, <code>makeRequiredValidator</code>	Aplicar	3 min
F-15	Partial en Clojure: <code>partial</code> nativo	Aplicar	3 min
F-16	Currying: concepto y tabla de diferencias con partial	Comprender	3 min
F-17	Currying en TS: <code>curry2</code> , <code>cHasMinLength</code> , pipeline de passwords	Aplicar	3 min
F-18	Currying en Clojure: <code>make-validator</code> , <code>validate-field</code>	Aplicar	3 min
F-19	<code>Result<T, E></code> : concepto, ventajas sobre try/catch	Comprender	3 min
F-20	Validación encadenada: <code>chain</code> , <code>validateForm</code> pipeline	Aplicar	4 min
F-21	Result en handler HTTP: Express/Hono sin try/catch	Aplicar	3 min
F-22	Middleware como HOF: concepto y patrón	Comprender	3 min
F-23	Middleware en TS: <code>withAuth</code> , <code>withLogging</code> , composición	Aplicar	4 min

Bloque 3 — Recursión de cola y patrones avanzados (30 min, F-24 a F-31)

Filmina	Contenido	Nivel Bloom	Tiempo est.
F-24	Recursión y stack overflow: visualización del problema	Comprender	3 min
F-25	Tail call optimization: acumulador, reemplazo de frame	Comprender	3 min
F-26	<code>recur</code> en Clojure: <code>sum-list</code> , <code>my-flatten</code>	Aplicar	4 min
F-27	Recursión de cola en TS: <code>sumList</code> , <code>findInTree</code>	Aplicar	4 min
F-28	Memoization: concepto, cuándo usar/no usar	Comprender	3 min
F-29	Memoization en TS y Clojure: implementación, <code>getConfig</code>	Aplicar	4 min
F-30	Lazy sequences: <code>range</code> , streaming de logs	Comprender	4 min
F-31	DSL de validación data-driven en Clojure	Analizar	5 min

Bloque 4 — Taller y cierre (20 min, F-32 a F-36)

Filmina	Contenido	Nivel Bloom	Tiempo est.
F-32	Tabla comparativa Clojure ↔ TypeScript	Analizar	3 min
F-33	Consigna del taller: pipeline de registro A (TS) + B (Clojure)	Crear	10 min
F-34	Checklist de patrones aprendidos	Evaluar	2 min
F-35	Guía de adopción: cuándo aplicar cada patrón	Evaluar	3 min
F-36	Cierre, nexos con Tema 05 (mónadas) y Tema 06 (efectos)	Recordar	2 min

5. Actividades y artefactos (derivados de F-33)

Actividad principal — Taller integrador (F-33, 10 min en clase)

Parte A — TypeScript:

- Definir `type FormData = { name, email, password, age }.`
- Implementar 3 validators que devuelvan `Result<FormData, string>.`
- Componer con `pipe` o `reduce` + `chain`.
- Manejar el Result en un handler HTTP sin excepciones.

Parte B — Clojure:

1. Definir `user-rules` como vector de mapas `{:field :pred :msg}`.
2. Implementar motor `validate` que devuelva todos los errores.
3. Agregar `memoize` a una función de lookup costosa.
4. Componer pipeline completo con `->>`.

Entrega: comparar ambas soluciones lado a lado e identificar patrones comunes.

6. Recursos clave

- Filminas aprobadas: `filminas.md` (F-01 a F-36) — fuente de verdad de la clase.
- Tema 03 (fundamentos FP): prerequisite directo.
- Tema 05 (mónadas en TS): continuación natural — `Result` es la mónada `Either`.
- Tema 06 (efectos y IO): extensión del manejo de side effects.
- Ref. académica: Sweller & Chen (2023), *Extending Cognitive Load Theory* — justifica los 4 bloques de ≤ 35 min.
- Ref. académica: Mayer & Fiorella, *Multimedia Learning Principles* — fundamenta separación concepto/ejemplo/código.
- Ref. taxonómica: Anderson & Krathwohl (2001), *Bloom Revised Taxonomy* — niveles usados en tabla de filminas.

7. Indicadores de éxito (mapeados a filminas)

Indicador	Filminas	Nivel Bloom
Crea una HOF que devuelve otra función	F-05, F-14, F-17	Aplicar
Explica la diferencia entre partial y currying	F-13, F-16	Comprender
Implementa recursión de cola con acumulador	F-26, F-27	Aplicar
Modela errores con <code>Result</code> sin excepciones	F-19, F-20, F-21	Aplicar
Construye middleware composable con <code>pipe</code>	F-22, F-23	Aplicar
Compara soluciones Clojure ↔ TS identificando patrones portables	F-32	Analizar
Resuelve el taller integrador completo	F-33	Crear

8. Riesgos y mitigaciones

Riesgo	Filminas afectadas	Mitigación
Alumnos se pierden en sintaxis Clojure	F-08, F-15, F-26, F-31	Cada ejemplo Clojure tiene su equivalente TS en la filmina siguiente
Confundir currying con partial	F-16	Tabla explícita de diferencias en F-16; ejercicio en F-17/F-18
<code>Result</code> se percibe como over-engineering	F-19	F-21 muestra su uso natural en un handler HTTP real
Stack overflow es abstracto	F-24	Visualización paso a paso de la pila en F-24
Taller no alcanza en 10 min	F-33	Diseñado para iniciar en clase y terminar en el TP

9. Notas para producción posterior

- `minuta.md` — una entrada por cada filmina (F-01 a F-36) con tiempo, guión docente y código.
- `guia-estudio.md` — material didáctico para el alumno con desarrollo teórico expandido y ejemplos paso a paso.
- `guiaprofesor.md` — guía autocontenida con profundidad teórica, citas textuales y plan de contingencia.
- `tp.md` — consignas trazables a filminas; tipo `repo` con autograding.
- `filminas.md` — NO TOCAR, ya aprobadas (F-01 a F-36).

Diseño alineado 1:1 con las 36 filminas aprobadas. Los 10 objetivos de aprendizaje cubren los 4 bloques y los niveles Bloom de Recordar a Crear.